

User Documentation

Games

Starfield Structures

Here you will find the structures of Starfield Animation (.af) and Skeleton Rig (.rig) file formats as well as the Animation Scene and Starfield specific Rig Package for UNIV rig

Namespaces

Starfield specific data is under the CALUMI::SFBGS namespace. External "C" functions are namespace agnostic and have unique symbols

Contents

- [SFBGS Animation Scene](#)
- [SFBGS Animation](#)
 - [SFBGS Animation Block](#)
 - [SFBGS Animation Header Flags](#)
 - [SFBGS Animation Entries](#)
- [SFBGS Skeleton Rig](#)
 - [SFBGS Skeleton Bone](#)
 - [SFBGS Skeleton Rig Enums](#)
 - [SFBGS Skeleton Rig Package](#)

AnimationScene

An animation scene groups animations together with a skeleton rig for conversions. This class is used solely by the dll and is not derived from any file formats. A scene can be directly converted to and from the universal, or "UNIV", format for easier processing in tools such as blender.

 A skeleton rig is required for proper conversion

class SFBGS::AnimationScene :
UNIV::Convertible

Member Types

Member	Type	Description
sceneName	Utilities::StringContainer	Default = "MyStarfieldAnimationScene" It is recommended to make this unique
animations	Utilities::VectorContainer<SFBGS::Animation>	List of uniquely named animations
rig	SFBGS::SkeletonRig	Skeleton Rig reference

Member Functions

Function	Arguments	Return Type	Description
(constructor)	none	none	Default
(constructor)	Utilities:: StringContainer sceneName or const char* sceneName	none	Creates a scene with the given name
AddAnimationToScene	SFBGS:: Animation & animation, bool overwrite = true	bool	Adds an animation to the scene, if overwrite is set to true the animation will replace any animation with the same name (lower case insensitive). Returns true if the animation was successfully added.
RemoveAnimationFromScene	Utilities:: StringContainer & sceneToRemove or const char* sceneToRemove or unsigned int* idx	bool	Removes an animation with the given name or index. Returns true if the animation was removed.

Inherited Member Functions

Function	Arguments	Return Type	Description
ConvertFromUniversalScene	UNIV::AnimationScene& input, (optional) SFBGS::SkeletonRig & rigReference	bool	Converts a universal animation scene instance from the UNIV (game agnostic) format to the SFBGS (Starfield) format. If a SFBGS::SkeletonRig is given, then that rig will be used for the conversion directly.
ConvertToUniversalScene	SFBGS::Animation& animation, bool overwrite = true	UNIV::AnimationScene	Converts the SFBGS (Starfield) animation scene and returns the contents as an animation scene in the UNIV (game agnostic) format

External "C" Type Functions

- ✓ You can pass a Utilities::StringContainer pointer to get any messages from the dll, however if you do not want this information, simply pass a nullptr

Function	Arguments	Return Type	Description
SaveAnimationSceneToSFBGSFormatC	UNIV::AnimationScene* scene, const wchar_t* directoryPath, Utilities::StringContainer* errorMessage	bool	Saves the contents of the given UNIV animation scene to the given directory path under the default file designations (file names being the animation names and the skeleton.rig file appearing in the [rig name] folder)
SaveAnimationSceneToSFBGSFormatPathOverrideC	UNIV::AnimationScene* scene, const wchar_t** directoryPathArray, size_t arraySize, Utilities::StringContainer* errorMessage	bool	Override of the function above where an array of cstyle wStrings are used as the final file paths.
SaveAnimationSceneToSFBGSFormatUsingRigReferenceC	UNIV::AnimationScene* scene, const wchar_t* directoryPath, const wchar_t* sfbgsRigPath, Utilities::StringContainer* errorMessage	bool	Saves the contents to a UNIV animation scene using an SFBGS rig reference instead of converting and using a rig from the UNIV scene instance
SaveAnimationSceneToSFBGSFormatUsingRigReferencePathOverrideC	UNIV::AnimationScene* scene, const wchar_t** directoryPathArray, size_t arraySize, const wchar_t* sfbgsRigPath, Utilities::StringContainer* errorMessage	bool	Override of the function above where an array of cstyle wStrings are used as the file paths

LoadAnimationSceneFromSFBGSFormatC	<pre>const wchar_t** filePathsArray, int numberOfFiles, Utilities::StringContainer* errorMessage</pre>	UNIV::AnimationScene*	Using the file paths provided, .af and .rig files are read and converted into a newly created UNIV animation scene
LoadAnimationSceneFromSFBGSFormatAndSaveToJsonC	<pre>const wchar_t** filePathsArray, int numberOfFiles, Utilities::StringContainer* errorMessage, const wchar_t* jsonOutputPath</pre>	UNIV::AnimationScene*	Same as above but with a JSON formatted UNIV animation scene file saved to the output path provided.
LoadSFBGSSkeletonRigFromFileC	<pre>const wchar_t* filePath, Utilities::StringContainer* errorMessage</pre>	UNIV::SkeletonRig*	Loads an skeleton.rig file into an SFBGS SkeletonRig object. Typically used internally as a reference for a scene.

Animation

Each animation file features a collection of "Animation Blocks" in which timeline sequences are applied to a certain bone index of a skeletal rig.

 Hint: All Starfield classes, structs, and functions should be found under the `CALUMI::SFBGS` namespace

`class SFBGS::Animation :` `CALUMI::Readable`

 The `SFBGS::Animation` struct contains the exact data read from the `.af` file format. However, additional members are added for convenience, such as file size and the file name. The members that are not from within the file data are marked as such: `fileSize`

Member Types

Member	Type	Size In File (bytes)	Description
fileSize	size_t	n/a	The size of the file being read in
animationFileName	Utilities::StringContainer	n/a	A compiler agnostic string storing the name of the file
_headerStart	float [9]	32	An unknown entry that appears to be blank in the first 2 float, a quaternion (4 floats), and a vector (3 floats) however this is unconfirmed
_headerFlags	HeaderFlags	4	Flags for file processing
_versionNumber	short	2	Always 0x05
_boneCount	unsigned short	2	Number of rig bones to be considered
_frameCount	unsigned short	2	Number of frames to be processed
_indexAtlasCounter	unsigned short	2	Default = 0x02, this count tells how many entries are in the Index Atlas
_unknownFillCount	unsigned short	2	TBD
_preambleOffset	unsigned short	2	TBD
_nZeroFloats	float [3]	12	TBD
_unknownSuffixFillFloats	Utilities::VectorContainer<float>	4 x N	TBD
_preamble	TBD	Varies	TBD

_indexAtlas	Utilities::VectorContainer<unsigned short>	2 x N	An atlas that informs the processor which bones are assigned to the animation blocks and which bones are skipped. The first entry is the indices to skip and the next is how many to consider, in order, this repeats. (for example if there are 5 blocks and 7 bones, an indexAtlas may look like {0,4,2,1} in which the blocks will have bone index assignments, in order, as 0,1,2,3,6,7)
animationBlocks	Utilities::VectorContainer<AnimationBlock>	6+	A block of four sequences that are applied to a specific bone. (Rotation, Translation, Scale, Priority)
animationSuffixBlocks	Utilities::VectorContainer<AnimationBlock>	6+	TBD

Inherited Member Functions

Function	Arguments	Return Type	Description
ReadFromFile	Utilities::PathContainer& inputFilePath or const wchar_t* inputFilePath	Utilities::ExpectedContainer<bool, FileError>	Processes a given .af file into the struct, assigning data directly
WriteToFile	Utilities::PathContainer& outputFilePath or const wchar_t* outputFilePath	Utilities::ExpectedContainer<Utilities::StringContainer, FileError>	Serializes the given class into an .af file at the path given

Member Functions

Function	Arguments	Return Type	Description
_SumIndices	Utilities::VectorContainer<unsigned short> inputVector, indexCountingSolution type	unsigned short	Returns a sum of the odd, even, or all entries in the vector. Primarily used to navigate the indexAtlas
_evaluateHeaderFlags	none	void	Sets flags in the header to appropriate values according to the data within the structure

IndexCountingSolution

`enum class SGBGS::IndexCountingSolution :
uint8_t`

Member Types

Name	Value
odd	0
even	1
all	2

HeaderFlags

struct SFBGS::HeaderFlags

Member Types

Byte 1

Member	Type	Description
firstEntry	bool : 1	TBD, appears false always but unconfirmed
shortKeyCounters	bool : 1	if True : AnimationBlock entry counters use 2 bytes instead of 1
shortKeyFrameEntries	bool : 1	if True: AnimationBlock keyframes use 2 bytes instead of 1
scalarSequenceFlag	bool : 1	if True: at least 1 scalar sequence counter is greater than 0
u1, u2, u3, u4	bool : 1 (each)	Always false, likely unused

Bytes 2-4

Member	Type	Description
c1	char	Unused
c2	char	Unused
c3	char	Unused

AnimationBlock

class SFBGS::**AnimationBlock**

In the animation block, rotation and translation sequences are broken up into three arrays of data. The first array are the keyframes to be considered for each entry. The next two arrays are high and low precision values, where the larger value (lower precision) features an RLE compression scheme. Each small value (high precision) is considered once while the larger value is repeated for a certain amount called out by it's count entry. Here we refer to each small value as **entry**, and each large value as **prefix**.

Persistent Member Types

Member	Type	Size In File (bytes)	Description
<code>_rotationCount</code>	<code>uint8_t</code> or <code>uint16_t</code> (in class as <code>unsigned short</code>)	1 or 2	This tells the processor how many rotation entries and their respective keyframes are featured
<code>_rotationPrefixCount</code>	<code>uint8_t</code> or <code>uint16_t</code> (in class as <code>unsigned short</code>)	1 or 2	This tells the processor how many rotation prefixes are featured. This must never be greater than <code>_rotationCount</code>
<code>_translationCount</code>	<code>uint8_t</code> or <code>uint16_t</code> (in class as <code>unsigned short</code>)	1 or 2	This tells the processor how many translation entries and their respective keyframes are featured
<code>_translationPrefixCount</code>	<code>uint8_t</code> or <code>uint16_t</code> (in class as <code>unsigned short</code>)	1 or 2	This tells the processor how many translation prefixes are featured. This must never be greater than <code>_translationCount</code>
<code>_scalarCount</code>	<code>uint8_t</code> or <code>uint16_t</code> (in class as <code>unsigned short</code>)	1 or 2	This tells the processor how many scalar entries are featured.
<code>_bonePriorityCount</code>	<code>uint8_t</code> or <code>uint16_t</code> (in class as <code>unsigned short</code>)	1 or 2	This tells the processor how many priority entries are featured. Usually 0 or 1 in vanilla files

Conditional Member Types

Key Frame Vectors

Member	Type	Size In File (bytes)	Description
rotationKeyFrames	Utilities::VectorContainer<unsigned short>	varies	Key frame list for each entry of size(_rotationCount)
translationKeyFrames	Utilities::VectorContainer<unsigned short>	varies	Key frame list for each entry of size(_translationCount)
scalarKeyFrames	Utilities::VectorContainer<unsigned short>	varies	Key frame list for each entry of size(_scalarCount)
bonePriorityKeyFrames	Utilities::VectorContainer<unsigned short>	varies	Key frame list for each entry of size(_priorityCount)

Entry Vectors

Member	Type	Size In File (bytes)	Description
rotationEntries	Utilities::VectorContainer<SFBGS::RotationEntry>	varies	Entry list of size(_rotationCount)
rotationPrefixEntries	Utilities::VectorContainer<SFBGS::RotationPrefix>	varies	Prefix list of size(_rotationPrefixCount)
translationEntries	Utilities::VectorContainer<SFBGS::TranslationEntry>	varies	Entry list of size(_translationCount)
translationPrefixEntries	Utilities::VectorContainer<SFBGS::TranslationPrefix>	varies	Prefix list of size(_translationPrefixCount)
scalarEntries	Utilities::VectorContainer<short>	varies	Entry list of size(_scalarCount)
bonePriorityEntries	Utilities::VectorContainer<uint8_t>	varies	Entry list of size(_priorityCount)

Member Functions

Function	Arguments	Return Type	Description
(constructor)	none	none	Default
(constructor)	Utilities::VectorContainer<char>&buffer, unsigned long long& addressIndex, const HeaderFlags & flags	none	Used internally to build from and advance a buffer
SerializeIntoBuffer	Utilities::VectorContainer<char>&buffer, unsigned long long& addressIndex, const HeaderFlags & flags	void	Used internally to write data to and advance a buffer

AnimationEntries

Rotation Scheme

Rotations are broken up into two values, the prefix and entry. The equation to combine two compressed values into the raw quaternion component can be done as so:

$$f(e, p, p_f) = \text{sqrt}(1/2) * 1/64 * p + \text{sqrt}(1/2) * 1/16384 * e * (8^{p_f})$$

Where e is the entry component, p is the prefix component, and p_f is the component flag from the prefix struct. The entry value is bitshifted by 3 (or multiplied by 8) if the respective flag is set to true. This is repeated for all 3 components (first, second, and third) in which a fourth component, the missing component, can be found as so:

$$f(f, s, t) = \text{sqrt}(1 - f^2 - s^2 - t^2)$$

Missing Quaternion Component	Assignments
x	f→y s→z t→w
y	f→x s→z t→w
z	f→x s→y t→w
w	f→x s→y t→z

struct SFBGS::RotationPrefix

i Operators == and != are defined, comparing each member except for count. This is to assist in compression when folding and unfolding a prefix vector

Member Types

Member	Type (4 bytes total)	Description
first	<code>int8_t : 7</code>	First quaternion entry
firstFlag	<code>bool : 1</code>	First entry flag
second	<code>int8_t : 7</code>	Second quaternion entry
secondFlag	<code>bool : 1</code>	Second entry flag
third	<code>int8_t : 7</code>	Third quaternion entry
thirdFlag	<code>bool : 1</code>	Third entry flag
count	<code>int8_t : 6</code>	Repetitions (using program oriented counting starting at 0)
missing	<code>int8_t : 2</code>	Missing quaternion component, knowing this will inform the processor in which component the entries are assigned. <i>Quaternion component order [x, y, z, w]</i>

`struct SFBGS::RotationEntry`

Member Types

Member	Type (3 bytes total)	Description
first	<code>int8_t</code>	First quaternion entry
second	<code>int8_t</code>	Second quaternion entry
third	<code>int8_t</code>	Third quaternion entry

Translation Scheme

Translation compression is more simple than rotations however there are still some factors to consider when converting the values. The prefix and entry values are scaled by precision multipliers called out in the skeleton rig to which the animation is applied. For this reason, among others, **skeleton rig references are required for converting translations.**

struct SFBGS::TranslationPrefix

 Operators == and != are defined, comparing each member except for count. This is to assist in compression when folding and unfolding a prefix vector

Member Types

Member	Type (8 bytes total)	Description
x	uint16_t	X direction translation prefix
y	uint16_t	Y direction translation prefix
z	uint16_t	Z direction translation prefix
count	uint16_t	Repetitions (using human oriented counting starting at 1)

struct SFBGS::TranslationEntry

Member Types

Member	Type (3 bytes total)	Description
x	<code>int8_t</code>	X direction translation entry
y	<code>int8_t</code>	Y direction translation entry
z	<code>int8_t</code>	Z direction translation entry

Scalar Scheme

short

Each scalar entry is on a scale of 5000 units, where 5000 is 1.0

- Can be negative
- Can theoretically range from *-32,768 to 32,767* or *-6.5536 to 6.5534*
- If present, the appropriate header flag MUST be set to true or the game will not only ignore the scalar sequence, but read it in as the next expected set of data with severe consequences

Priority

`uint8_t`

Not much is known about this entry except that values seen in the vanilla game files range from 20-90 in increments of 10. At present it is believe to be used when additive animations are combined, taking the higher priority block for animating a rig's bone

SkeletonRig

SFBGS::SkeletonRigs contain a series of bones in a specific order. While vertex groups for meshes typically apply to a bone by name, animation blocks apply to a bone by index.



Hint: All Starfield classes, structs, and functions should be found under the `CALUMI::SFBGS` namespace

struct SFBGS::SkeletonRig :
CALUMI::ReadWritable

Member Types

Member	Type	Size In File (bytes)	Description
versionNumber	int	4	The file's version number
fileSize	unsigned int	4	The size of the file being read in
headerSize	unsigned int	4	Size, in bytes, of the header
padding1	n/a	4	Always empty
boneMapOffset	unsigned int	4	Offset leading to the bone map
padding2	n/a	2	Always empty
mathcingThree	uint64_t [3]	24	TBD
lowPrecision	float	4	High precision scale to be applied to transformation sequences in animations for this rig
highPrecision	float	4	Low precision scale to be applied to transformation sequences in animations for this rig
boneCount	uint16_t	2	Number of bones in this rig
boneCount_Animated	uint16_t	2	Number of bones in this rig, not including twist bones
padding3	n/a	4	Always empty
endOfHeader	uint8_t [16]	16	Unknown

boneEntries	Utilities::VectorContainer<SFBGS::SkeletonBone>	96 x N	List of bone entries in proper index order. The root bone is always the 0th index
boneMapArray	uint16_t [157]	314	A bone map that stores the index of the bone to be marked as that enum value (see SFBGS::BoneMapKey) This helps the game quickly find a specific bone such as COM, left_foot, etc.
stringArray	Utilities::VectorContainer<Utilities::StringContainer>	varies	Array of strings that are matched to a bone. The offset of each string is stored by the bone itself, however the strings typically share the exact index order of the bones

Inherited Member Functions

Function	Arguments	Return Type	Description
ReadFromFile	Utilities::PathContainer& inputFilePath or const wchar_t* inputFilePath	Utilities::ExpectedContainer<bool, FileError>	Processes a given .rig file into the struct, assigning data directly
WriteToFile	Utilities::PathContainer& outputFilePath or const wchar_t* outputFilePath	Utilities::ExpectedContainer<Utilities::StringContainer, FileError>	Serializes the given class into an .rig file at the path given

Member Functions

Function	Arguments	Return Type	Description
IsMarkedMannequin	none	bool	Retruns if any of the bones have a certain mannequin trait, only found on the mannequin rigs
ConvertToSFBGSRig	UNIV::SkeletonRig& inputRig	SFBGS::SkeletonRig	Returns an SFBGS skeleton rig converted from a UNIV skeleton rig
ConvertToUniversalRig	SFBGS::SkeletonRig & inputRig	UNIV::SkeletonRig	Returns a UNIV skeleton rig converted from an SFBGS skeleton rig

SkeletonBone

```
struct SFBGS::SkeletonBone
```

Member	Type	Size In File (bytes)	Description
localRotation	Math:: Quaternion	16	Quaternion that represents the rotation relative to its parent bone
globalRotation	Math:: Quaternion	16	Quaternion that represents the rotation relative to its rig origin
position	Math:: Vector3	12	Vector3 that represents the position of the bone relative to its parent bone
boneType	BoneType	4	-1 or 1 for identifying root bones
nameOffset	uint64_t	8	offset pointing to the bone's name string
parentBoneIndex	int32_t	4	index of this bone's parent bone
twistDriverMqnIndex	int32_t	4	usually -1, on mannequin rigs where this bone is a twist bone this value is the parent bone index
twistDriverIndex	int32_t	4	Twist bone influence, -1 if not a twist bone

padding1	<code>int32_t</code>	4	always -1 (since skipped entries are entered as -1 and null values are usually padding this could be an unused member that is simply skipped in all files)
mirrorBoneIndex	<code>int32_t</code>	4	The index points to this bone's mirrored bone. If no mirror is entered this should be equal to this bone's index.
term05	<code>int32_t</code>	4	TBD
twistDriverWeight	<code>float</code>	4	How much influence the twist driver has on this bone's rotation
padding2	<code>int32_t</code>	4	0
unknownScalar	<code>float</code>	4	TBD
term08	<code>int32_t</code>	4	TBD

Function	Arguments	Return Type	Description
GetBoneTypeAsUNIVEnum	none	UNIV:: <code>BoneType</code>	Gets this bone's type in the form of the UNIV bone type enum
GetBoneTypeAsString	none	<code>const char*</code>	Gets this bone's type as a c style string

Enums

enum class SFBGS::BoneType : uint32_t

Enum Values

Name	Value
Default	1
Twist	-1

enum class SFBGS::BoneMapKey: uint8_t

Enum Values

Name	Value
Root	0
AnimObjectA	1
AnimObjectB	2
AnimObjectC	3
AnimObjectD	4
Camera	5
Camera_Control	6
CamTargetParent	7
CameraTarget	8
COM	9
C_Hips	10
R_Thigh	11
R_Calf	12
R_Foot	13
R_Toe	14
L_Thigh	19
L_Calf	20
L_Foot	21
L_Toe	22
C_Spine	29
C_Spine1	30
C_Spine2	31
C_Chest	32
C_Neck	33

C_Neck1	34
C_Head	35
L_Clavicle	39
L_Biceps	40
L_Forearm	41
L_Wrist	42
L_Thumb	43
L_Thumb1	44
L_Thumb2	45
L_Cup	46
L_Pinky	47
L_Pinky1	48
L_Pinky2	49
L_Ring	50
L_Ring1	51
L_Ring2	52
L_Middle	53
L_Middle1	54
L_Middle2	55
L_Index	56
L_Index1	57
L_Index2	58
L_AnimObject1	59
L_AnimObject2	60
L_AnimObject3	61

L_Arm	62
L_Elbow	64
C_BackPack	68
C_BackPackHose	69
R_Clavicle	72
R_Biceps	73
R_Forearm	74
R_Wrist	75
R_Thumb	76
R_Thumb1	77
R_Thumb2	78
R_Cup	79
R_Pinky	80
R_Pinky1	81
R_Pinky2	82
R_Ring	83
R_Ring1	84
R_Ring2	85
R_Middle	86
R_Middle1	87
R_Middle2	88
R_Index	89
R_Index1	90
R_Index2	91
R_AnimObject1	92

R_AnimObject2	93
R_AnimObject3	94
R_Arm	95
R_Elbow	97
Weapon	101
Bolt01	119
Bolt02	120
WeaponLeft	134
DirectAt	137
C_Waist	140
Camera_Control_FP	141
R_Thigh_Twist	142
R_Thigh_Twist1	143
L_Thigh_Twist	144
L_Thigh_Twist1	145
C_Neck_Twist	146
L_Wrist_Twist	147
L_Wrist_Twist1	148
L_Wrist_Twist2	149
L_Biceps_Twist	150
L_Biceps_Twist1	151
R_Wrist_Twist	152
R_Wrist_Twist1	153
R_Wrist_Twist2	154
R_Biceps_Twist	155

R_Biceps_Twist1	156
None (Only used by the dll as the source array is only 157 entries in size)	255

SFBGS_RigPackage

Constants

constexpr int SFBGSMAPSIZE = 157

static const char* SFBGS_RIG_PACKAGE =
"SFBGS_RIG_PACKAGE"

Standard Precision Values

Precision Type	High Value	Low Value
Default	1/4000 (0.00025f)	1/32 (0.03125f)
First Person	1/16000 (0.0000625f)	1/128 (0.0078125f)
Ship	1/500 (0.002f)	1/4 (0.25f)

struct SFBGS::SFBGS_RigPackage :
UNIV::RigPackage

The SFBGS Rig Package can be applied to a UNIV Skeleton Rig to carry data and definitions that apply only when the UNIV rig is converted to Starfield. This is done to prevent the UNIV and SFBGS definitions from becoming entangled.

The Rig Package is not found in the Starfield file format and is solely used by the dll to handle data in a universal manner. Imagine if we had our skeleton rig and wanted to convert it to both Starfield AND say... The Elder Scrolls VI... We would want a different set of rig data that only matters for TES6 without making (even more of) a mess of our code

Member Types

Member	Type	Description
highPrecisionValue	float	This tells the converter what high precision value to assign to the SFBGS rig when converted
lowPrecisionValue	float	This tells the converter what low precision value to assign to the SFBGS rig when converted
isMannequin	bool	This marks whether the rig is intended to be a mannequin in game
rigMap	Utilities::StringContainer [SFBGSRIGMAPSIZE]	This stores a map of bone names assigned to a specific id for the rig so that Starfield can quickly process bone information. Each index represents the numerical value of the BoneMapKey

Member Functions

Function	Arguments	Return Type	Description
GetPackageType	none	<code>const char*</code>	Returns SFBGS_RIG_PACKAGE constant. Used when casting a RigPackage pointer or detecting if any of the UNIV rig's assigned packages are of this child type (there is only one allowed on each UNIV rig)
BoneIsMapped	<code>const char*</code> boneName	<code>bool</code>	Returns true if a bone exists in the map
KeyIsMapped	<code>BoneMapKey</code> key	<code>bool</code>	Returns true if a key has a bone name assigned to it
AddBoneToMap	<code>BoneMapKey</code> key, <code>const char*</code> boneName, <code>bool</code> overwrite = true	<code>bool</code>	Adds a bone to a map by name (will use the bone's name string), returns true if the bone was successfully added. When overwrite is set to false, any entries with a matching key or string will cancel the function and return false.

AddBoneToMap	<code>BoneMapKey</code> key, UNIV:: <code>SkeletonBone</code> & bone, <code>bool</code> overwrite = <code>true</code>	<code>bool</code>	Adds a bone to a map by instance (will use the bone's name string), returns true if the bone was successfully added. When overwrite is set to false, any entries with a matching key or string will cancel the function and return false.
RemoveBoneFromMap	<code>BoneMapKey</code> key <i>or</i> <code>const char*</code> boneName	<code>bool</code>	Removes a bone from the map, returns false if no bone was found
GetBoneKey	<code>const char*</code> boneName	<code>BoneMapKey</code>	Returns the key, if any, that this bone is assigned to
GetBoneNameFromKey	<code>BoneMapKey</code> key	<code>const char*</code>	Returns the name of the bone that is assigned to this key
SetPrecisionValues	<code>PrecisionSet</code> setting, (optional) <code>float</code> custom1 = SFBGSDefaultPrecision.first, (optional) <code>float</code> custom2 = SFBGSDefaultPrecision.second	<code>void</code>	Sets the (SFBGS specific) precision values for this rig. Custom values are auto sorted by size and are only used if the <code>PrecisionSet</code> enum is set to Custom

Global Functions

Function	Arguments	Return Type	Description
CreateNewSFBGSRigPackage	UNIV::SkeletonRig& rig, bool overwrite = true	SFBGS_RigPackage *	Creates a new SFBGS_RigPackage on the given UNIV::SkeletonRig. Returns a pointer to the newly created package for convenience.
RemoveSFBGSRigPackage	UNIV::SkeletonRig& rig	bool	Removes the SFBGS_RigPackage from the given UNIV::SkeletonRig. Returns true if the package was successfully removed.

External "C" Style Functions



You can pass a Utilities::StringContainer pointer to get any messages from the dll, however if you do not want this information, simply pass a nullptr

Function	Arguments	Return Type	Description
SFBGSRigPackage_AddPackageToSkeletonRigC	UNIV::SkeletonRig* rig, Utilities::StringContainer* errorMessage, bool overwrite	bool	Creates an SFBGS package on the given UNIV skeleton rig. If one already exists and overwrite is set to false, the existing package will take priority and it will return false.
SFBGSRigPackage_RemoveRigPackageFromSkeletonRigC	UNIV::SkeletonRig* rig, Utilities::StringContainer* errorMessage	bool	Returns true if an SFBGS rig package is removed
SFBGSRigPackage_BoneIsMappedC	UNIV::SkeletonRig* rig, const char* boneName	bool	Returns true if an SFBGS rig package exists and this bone is mapped on it
SFBGSRigPackage_KeyIsMappedC	UNIV::SkeletonRig* rig, uint8_t key	bool	Returns true if an SFBGS rig package exists and a bone is assigned to this key
SFBGSRigPackage_AddBoneNameToMapC	UNIV::SkeletonRig* rig, uint8_t key, const char* boneName, Utilities::StringContainer* errorMessage, bool overwrite	bool	Returns true if the boneName was added successfully to the rig's SFBGS rig package

SFBGSRigPackage_ AddBoneToMapC	UNIV:: SkeletonRig* rig, uint8_t key, UNIV:: SkeletonBone & bone, Utilities:: StringContainer* errorMessage, bool overwrite	bool	Returns true if the bone (name string) was added successfully to the rig's SFBGS rig package
SFBGSRigPackage_ RemoveBoneFromMapUsingKeyC	UNIV:: SkeletonRig* rig, uint8_t key	bool	Returns true if a bone mapped to this key was removed from the rig's SFBGS rig package map
SFBGSRigPackage_ RemoveBoneFromMapUsingNameC	UNIV:: SkeletonRig* rig, const char* boneName	bool	Returns true if this bone was removed from the rig's SFBGS rig package map
SFBGSRigPackage_ GetBoneKeyC	UNIV:: SkeletonRig* rig, const char* boneName	uint8_t	Returns the key in numeric form that this bone is mapped to or none if no bone under this name is mapped
SFBGSRigPackage_ GetBoneNameFromKeyC	UNIV:: SkeletonRig* rig, uint8_t key	const char*	Returns the name of the bone mapped to this key, returns "" if no bone is mapped
SFBGSRigPackage_ SetMannequinC	bool isMannequin	bool	Returns the isMannequin boolean
SFBGSRigPackage_ GetRigMapSize	none	size_t	Convenient way to get the size of the map

SFBGSRigPackage_ SetPrecisionToDefaultC	UNIV:: SkeletonRig * rig	void	Sets the package precision values to default
SFBGSRigPackage_ SetPrecisionToFirstPersonC	UNIV:: SkeletonRig * rig	void	Sets the package precision values to first person
SFBGSRigPackage_ SetPrecisionToShipValuesC	UNIV:: SkeletonRig * rig	void	Sets the package precision values to starship
SFBGSRigPackage_ SetPrecisionToCustomC	UNIV:: SkeletonRig * rig, float custom1, float custom2	void	Sets the package precision values to custom with two custom floats. The custom values are auto applied to the high and low precision. It is not recommended to do this unless the user is aware that the results are not guaranteed to work as expected

MainIO

Universal Structures

Universal structs and classes act as the main interface for this DLL. Designed to be simple to add and modify data, the goal is to easily expand upon this system when new games and format are released

Namespaces

Universal, non-game specific data is under the CALUMI::UNIV namespace. External "C" functions are namespace agnostic and have unique symbols

AnimationScene

An animation scene groups animations together with a skeleton rig for conversions. A scene can be directly converted to and from supported game formats.

 A skeleton rig is required for proper conversion

class UNIV::AnimationScene

Member Types

Member	Type	Description
sceneName	Utilities::StringContainer	Default = "MyAnimationScene" It is recommended to make this unique
animations	Utilities::VectorContainer<UNIV::Animation>	List of uniquely named animations
rig	UNIV::SkeletonRig	Skeleton Rig reference

Member Functions

Function	Arguments	Return Type	Description
(constructor)	none	none	Default
(constructor)	Utilities:: StringContainer sceneName or const char* sceneName	none	Creates a scene with the given name
AddAnimationToScene	UNIV:: Animation & animation, bool overwrite = true	bool	Adds an animation to the scene, if overwrite is set to true the animation will replace any animation with the same name (lower case insensitive). Returns true if the animation was successfully added.
RemoveAnimationFromScene	Utilities:: StringContainer & sceneToRemove or const char* sceneToRemove or unsigned int* idx	bool	Removes an animation with the given name or index. Returns true if the animation was removed.

External "C" Type Functions



You can pass a Utilities::[StringContainer](#) pointer to get any messages from the dll, however if you do not want this information, simply pass a nullptr

Function	Arguments	Return Type	Description
CreateAnimationSceneC	<code>const char* sceneName</code>	<code>UNIV::AnimationScene*</code>	Returns a new, heap allocated , UNIV AnimationScene
AddRigToAnimationSceneC	<code>UNIV::AnimationScene* scene,</code> <code>UNIV::SkeletonRig* rig,</code> <code>Utilities::StringContainer* errorMessage</code>	<code>bool</code>	Copies rig data from a heap allocated UNIV SkeletonRig to the animation scene, automatically deletes the heap allocated rig data .
AddAnimationToAnimationSceneC	<code>UNIV::AnimationScene* scene,</code> <code>UNIV::Animation* animation,</code> <code>bool overwrite,</code> <code>Utilities::StringContainer* errorMessage</code>	<code>bool</code>	Copies animation data from a heap allocated UNIV Animation to the animation scene, automatically deletes the heap allocated rig data .
DeleteAnimationSceneC	<code>UNIV::AnimationScene* ptr</code>	<code>bool</code>	Deletes/Frees the memory claimed by the heap allocated UNIV AnimationScene
GetAnimationC	<code>UNIV::AnimationScene* source,</code> <code>int index,</code> <code>Utilities::StringContainer* errorMessage</code>	<code>UNIV::Animation*</code>	Returns a pointer to the UNIV Animation at the given index of the source UNIV AnimationScene
GetAnimationCountC	<code>UNIV::AnimationScene* source</code>	<code>size_t</code>	Returns the count of UNIV Animation's in this UNIV AnimationScene
GetAnimationSceneNameC	<code>UNIV::AnimationScene* source</code>	<code>const char*</code>	Returns the UNIV AnimationScene name

GetSkeletonRigC	UNIV:: AnimationScene * source	UNIV:: SkeletonRig *	Returns a pointer to this UNIV AnimationScene's UNIV SkeletonRig
HasSkeletonRigC	UNIV:: AnimationScene * source	bool	Detects if the UNIV AnimationScene has a valid UNIV SkeletonRig

Animation

Each animation file features a collection of "Animation Blocks" in which timeline sequences are applied to a certain bone index of a skeletal rig.

 Hint: All Universal classes, structs, and functions should be found under the `CALUMI::UNIV` namespace

`class UNIV::Animation :` `CALUMI::Readable`

Member Types

Member	Type	Description
animationTitle	Utilities::StringContainer	Unique name for this animation Default = "NO TITLE"
animationBlocks	Utilities::VectorContainer<AnimationBlock>	A block of four sequences that are applied to a specific bone. (Rotation, Translation, Scale, Priority)

Member Functions

Function	Arguments	Return Type	Description
(Constructor)	none	none	Default
(Constructor)	<pre>const Utilities::StringCont ainer& title, unsigned int initialBlockCount = 8</pre>	void	Constructs an animation with the given title and starts with a certain amount of space reserved for the count entered (reserved space is not filled with animation block but optimizes memory for adding them later)

External "C" Style Functions

 Be mindful to track and handle memory allocation

Animation Interface

Function	Arguments	Return Type	Description
CreateAnimationC	<code>const char*</code> animationTitle, <code>unsigned int</code> rigBoneCount	UNIV:: <code>Animation*</code>	Returns a newly heap allocated UNIV Animation.
GetAnimationBlockC	UNIV:: <code>Animation*</code> source, <code>int</code> index, Utilities:: <code>StringContainer*</code> errorMessage	UNIV:: <code>AnimationBlock*</code>	Returns the UNIV AnimationBlock at the given index from this UNIV Animation's block list.
GetAnimationBlockCountC	UNIV:: <code>Animation*</code> source	<code>size_t</code>	Gets the size of this UNIV Animation's block list.
GetAnimationTitleC	UNIV:: <code>Animation*</code> source	<code>const char*</code>	Get the title of this UNIV Animation.
GetFrameCountC	UNIV:: <code>Animation*</code> source	<code>size_t</code>	Gets the total frame count on this UNIV Animation. If the final frame on this animation is 30, then 31 will be returned as this would include frame 0.
DeleteAnimationC	UNIV:: <code>Animation*</code> source	<code>bool</code>	Frees the heap allocated memory used by this UNIV Animation.
AddAnimBlockToAnimationC	UNIV:: <code>Animation*</code> source, UNIV:: <code>AnimationBlock*</code> blockToAdd, <code>bool</code> overwrite, Utilities:: <code>StringContainer*</code> errorMessage	<code>bool</code>	Copies a given UNIV AnimationBlock to the UNIV Animation. The original, heap allocated, UNIV AnimationBlock is deleted.

AnimationBlock

struct UNIV::AnimationBlock

In the animation block, sequences are applied to a given bone index.

Member Types

Member	Type	Description
boneIndex	int	The bone index assigned to this animation block
boneName	Utilities::StringContainer	The name of the bone assigned to this animation block (mostly used for tracking purposes)
_rotationSequence	Utilities::VectorContainer<Rotation>	Sequence of rotation entries
_translationSequence	Utilities::VectorContainer<Translation>	Sequence of translation entries
_scalarSequence	Utilities::VectorContainer<Scalar>	Sequence of scalar entries
_prioritySequence	Utilities::VectorContainer<Priority>	Sequence of priority entries (typically a single entry for additive animations or none)

Member Functions

Function	Arguments	Return Type	Description
(constructor)	none	none	Default
AddRotationEntry, AddTranslationEntry, AddScalarEntry, AddPriorityEntry	UNIV::Rotation& input or UNIV::Translation& input or , bool overwrite = true UNIV::Scalar& input or UNIV::Priority& input	bool	Returns true if the respective entry was successfully added. Will overwrite if a matching frame is found and the overwrite boolean is set to true.
RemoveRotationEntry, RemoveTranslationEntry, RemoveScalarEntry, RemovePriorityEntry	unsigned int frame	bool	Returns true if the entry with this frame was successfully removed.
ClearRotationEntries, ClearTranslationEntries, ClearScalarEntries, ClearPriorityEntries	none	void	Clears the sequence of all entries
GetRotationEntryCount, GetTranslationEntryCount, GetScalarEntryCount, GetPriorityEntryCount	none	size_t	Returns the entry count for the respective sequence

External "C" Style Functions

⚠ Be mindful to track and handle memory allocation

Animation Block Interface

Function	Arguments	Return Type	Description
CreateAnimBlockC	<code>const char*</code> boneName, <code>int</code> boneIndex, <code>Utilities::StringContainer*</code> errorMessage	UNIV::AnimationBlock*	Returns a newly heap allocated UNIV AnimationBlock
DeleteAnimationBlockC	UNIV::AnimationBlock* ptr	<code>bool</code>	Frees the heap allocated memory used by this UNIV Animation
GetAnimBlockBoneNameC	UNIV::AnimationBlock* source	<code>const char*</code>	Get's the name of this UNIV AnimationBlock
GetAnimBlockBoneIndexC	UNIV::AnimationBlock* source	<code>int</code>	Returns the boneIndex assigned to this UNIV AnimationBlock
GetLastFrameInAnimationBlockC	UNIV::AnimationBlock* source	<code>unsigned int</code>	Returns the final frame as it is in the sequence collection

Animation Entry Interface

Rotations

Function	Arguments	Return Type	Description
AddRotationSqToAnimationBlockC	UNIV::AnimationBlock* block, UNIV::Rotation* rotSq, unsigned int size, bool overwrite	bool	Adds a given UNIV Rotation entry array of the given size to the sequence on the given UNIV AnimationBlock. To add a single entry, simply set the size to 1.
GetRotationSqArrayC	UNIV::AnimationBlock* source	UNIV::Rotation*	Returns a pointer to the first entry of the sequence
GetRotationFromSqC	UNIV::AnimationBlock* source, int index, Utilities::StringContainer* errorMessage	UNIV::Rotation*	Returns the entry at a given index.
GetRotationSqSizeC	UNIV::AnimationBlock* source	size_t	Returns the size of the sequence. Can be used when handling the returned array.

Translations

Function	Arguments	Return Type	Description
AddTranslationSqToAnimBlockC	UNIV::AnimationBlock* block, UNIV::Translation* trnSq, unsigned int size, bool overwrite	bool	Adds a given UNIV Translation entry array of the given size to the sequence on the given UNIV AnimationBlock. To add a single entry, simply set the size to 1.
GetTranslationSqArrayC	UNIV::AnimationBlock* source	UNIV::Translation*	Returns a pointer to the first entry of the sequence
GetTranslationFromSqC	UNIV::AnimationBlock* source, int index, Utilities::StringContainer* errorMessage	UNIV::Translation*	Returns the entry at a given index.
GetTranslationSqSizeC	AnimationBlock* source	size_t	Returns the size of the sequence. Can be used when handling the returned array.

Scalar

Function	Arguments	Return Type	Description
AddScalarSqToAnimBlockC	UNIV::AnimationBlock* block, UNIV::Scalar* sclrSq, unsigned int size, bool overwrite	bool	Adds a given UNIV Scalar entry array of the given size to the sequence on the given UNIV AnimationBlock. To add a single entry, simply set the size to 1.
GetScalarSqArrayC	UNIV::AnimationBlock* source	UNIV::Scalar*	Returns a pointer to the first entry of the sequence
GetScalarFromSqC	UNIV::AnimationBlock* source, int index, Utilities::StringContainer* errorMessage	UNIV::Scalar*	Returns the entry at a given index.
GetScalarSqSizeC	UNIV::AnimationBlock* source	size_t	Returns the size of the sequence. Can be used when handling the returned array.

Priorities

Function	Arguments	Return Type	Description
AddPrioritySqToAnimBlockC	UNIV::AnimationBlock* prtySq, UNIV::Priority* rotSq, unsigned int size, bool overwrite	bool	Adds a given UNIV Priority entry array of the given size to the sequence on the given UNIV AnimationBlock. To add a single entry, simply set the size to 1.
GetPrioritySqArrayC	UNIV::AnimationBlock* source	UNIV::Priority*	Returns a pointer to the first entry of the sequence
GetPriorityFromSqC	UNIV::AnimationBlock* source, int index, Utilities::StringContainer* errorMessage	UNIV::Priority*	Returns the entry at a given index.
GetPrioritySqSizeC	UNIV::AnimationBlock* source	size_t	Returns the size of the sequence. Can be used when handling the returned array.

AnimationEntries

struct UNIV::Translation

A UNIV Translation stores key frame data for modifying a rig bone's position

Member Types

Member	Type	Description
frame	<code>uint16_t</code>	The frame for this entry. The sequence begins with frame 0
translation	<code>Math::Vector3D</code>	The Vector3 (double components) representing the translation for this entry

Member Functions

Function	Arguments	Return Type	Description
(constructor)	none	none	Default
(constructor)	<code>uint16_t</code> frame, <code>Math::Vector3D&</code> translation	none	-
operators [<code><</code> , <code><=</code> , <code>></code> , <code>>=</code> , <code>==</code> , <code>!=</code>]	<code>UNIV::Translation&</code> other	<code>bool</code>	Compares the frame of the two entries

struct UNIV::Rotation

A UNIV Rotation stores key frame data for modifying a rig bone's rotation

Member Types

Member	Type	Description
frame	<code>uint16_t</code>	The frame for this entry. The sequence begins with frame 0
rotation	<code>Math::Quaternion</code>	The rotation representation for this entry

Member Functions

Function	Arguments	Return Type	Description
(constructor)	none	none	Default
(constructor)	<code>uint16_t</code> frame, <code>Math::Quaternion</code> & rotation	none	-
operators [<code><</code> , <code><=</code> , <code>></code> , <code>>=</code> , <code>=</code> , <code>!=</code>]	<code>UNIV::Rotation</code> & other	<code>bool</code>	Compares the frame of the two entries

`struct UNIV::Scalar`

A UNIV Scalar stores key frame data for modifying a rig bone's scale

Member Types

Member	Type	Description
frame	<code>uint16_t</code>	The frame for this entry. The sequence begins with frame 0
scalar	<code>float</code>	The scalar representation for this entry (1.0 being the original state). Can be negative.

Member Functions

Function	Arguments	Return Type	Description
(constructor)	none	none	Default
(constructor)	<code>uint16_t</code> frame, <code>float</code> scalar	none	-
operators [<code><</code> , <code><=</code> , <code>></code> , <code>>=</code> , <code>==</code> , <code>!=</code>]	<code>UNIV::Scalar&</code> other	<code>bool</code>	Compares the frame of the two entries

`struct UNIV::Priority`

⚠ Not much is known about priority yet and it may come with unexpected results in game. It is advised to only enter a priority on the entry frame

A UNIV Priority stores key frame data for a rig bone's sequence data. A higher priority *should* be used when two sequences for one block are played at the same time in the case of additive animations. This behavior is not confirmed in any of the games supported.

Member Types

Member	Type	Description
frame	<code>uint16_t</code>	The frame for this entry. The sequence begins with frame 0
priority	<code>uint8_t</code>	Default = 90

Member Functions

Function	Arguments	Return Type	Description
(constructor)	none	none	Default
(constructor)	<code>uint16_t</code> frame, <code>uint8_t</code> priority	none	-
operators [<code><</code> , <code><=</code> , <code>></code> , <code>>=</code> , <code>=</code> , <code>!=</code>]	UNIV:: <code>Priority</code> & other	<code>bool</code>	Compares the frame of the two entries

External "C" Style Functions

 Be mindful to track and handle memory allocation

Entry Agnostic

Function	Arguments	Return Type	Description
GetFrameFromEntry C	<code>void*</code> source	<code>uint16_t</code>	Returns the frame from the given entry pointer. If a non UNIV AnimationEntry, the first 2 bytes of the given pointer's data will be returned, which may have unexpected behavior.

Rotations

Function	Arguments	Return Type	Description
CreateRotationEntry C	<code>uint16_t</code> frame, <code>float</code> x, <code>float</code> y, <code>float</code> z, <code>float</code> w	UNIV:: <code>Rotation</code> *	Returns a new, heap allocated , entry
DeleteRotationEntry C	UNIV:: <code>Rotation</code> * ptr	<code>bool</code>	Frees the memory of the entry
GetValueFromRotationEntryC	UNIV:: <code>Rotation</code> * source	Math:: <code>Quaternion</code> *	Returns a pointer to the rotation's quaternion

Translations

Function	Arguments	Return Type	Description
CreateTranslationEntryC	uint16_t frame, double x, double y, double z	UNIV::Translation*	Adds a given UNIV Translation entry array of the given size to the sequence on the given UNIV AnimationBlock. To add a single entry, simply set the size to 1.
DeleteTranslationEntryC	UNIV::Translation* ptr	bool	Returns a pointer to the first entry of the sequence
GetValueFromTranslationEntryC	UNIV::Translation* source	Math::Vector3D*	Returns the entry at a given index.

Scalar

Function	Arguments	Return Type	Description
CreateScalarEntryC	uint16_t frame, float scalar	UNIV::Scalar*	Adds a given UNIV Scalar entry array of the given size to the sequence on the given UNIV AnimationBlock. To add a single entry, simply set the size to 1.
DeleteScalarEntryC	UNIV::Scalar* ptr	bool	Returns a pointer to the first entry of the sequence
GetValueFromScalarEntryC	UNIV::Scalar* source	float	Returns the entry at a given index.

Priorities

Function	Arguments	Return Type	Description
CreatePriorityEntryC	<code>uint16_t</code> frame, <code>uint8_t</code> priority	UNIV::Priority*	Adds a given UNIV Priority entry array of the given size to the sequence on the given UNIV AnimationBlock. To add a single entry, simply set the size to 1.
DeletePriorityEntryC	UNIV::Priority* ptr	<code>bool</code>	Returns a pointer to the first entry of the sequence
GetValueFromPriorityEntryC	UNIV::Priority* source	<code>uint8_t</code>	Returns the entry at a given index.

SkeletonRig

UNIV::SkeletonRigs contain a series of bones in a specific order. While vertex groups for meshes typically apply to a bone by name, animation blocks apply to a bone by index.



Hint: All Universal classes, structs, and functions should be found under the [CALUMI::UNIV](#) namespace

struct UNIV::SkeletonRig

Member Types

Member	Type	Description
rigName	Utilities::StringContainer	The rig's name
boneEntries	Utilities::VectorContainer<UNIV::SkeletonBone>	Collection of bone definitions
rigPackageManager	UNIV::RigPackageManager	Stores package information that defines behavior on a game by game basis allowing the rig itself to stay universal as an entity

Member Functions

Function	Arguments	Return Type	Description
ValidateNames	none	Utilities::ExpectedContainer< bool, Utilities::StringContainer>	Returns an expected type, with an errorMessage if has_value() is false. This function determines if all of the rig's bones have unique, case insensitive, names.
ValidateParentIndices	none	Utilities::ExpectedContainer< bool, Utilities::StringContainer>	Returns an expected type, with an errorMessage if has_value() is false. This function determines if all of the rig's bones have a valid parent index assigned.
AddBoneToRig	Math::Quaternion rotation, Math::Vector3 position, Utilities::StringContainer boneName, Utilities::StringContainer parentName <i>or</i> int parentIndex, bool localValues = true	bool	Returns true if the bone is successfully added.

RenameBone	<code>const char*</code> <code>boneName</code> <i>or</i> <code>size_t</code> <code>boneIndex</code> <code>const char*</code> <code>newName</code>	<code>bool</code>	The preferred way to rename an existing bone that exists in a UNIV SkeletonRig. Will also send the appropriate information to the rig packages on this UNIV SkeletonRig
CreateBoneMirrorPair	<code>int</code> <code>i1</code> , <code>int</code> <code>i2</code>	<code>bool</code>	Pairs two bone indices to mirror each other
ResetAllBoneMirrors	<code>none</code>	<code>bool</code>	Sets all mirror indices
VerifyExclusiveBoneMirrors	<code>none</code>	<code>bool</code>	Returns true if any bone with a mirror index declared, has that mirror bone point back to it
GetAnimatedBoneCount	<code>none</code>	<code>size_t</code>	Returns the amount of default bone types (Twist bones, for example, are not typically animated and are not counted)
GetBoneCount	<code>none</code>	<code>size_t</code>	Returns the total bone count
GetBoneIndex	<code>Utilities::StringContainer</code> <code>boneName</code>	<code>Utilities::ExpectedContainer<size_t,</code> <code>Utilities::StringContainer></code>	Returns an expected type with either the index of the bone entered or an error message if <code>has_value()</code> is false.

External "C" Type Functions

Skeleton Rig Interface

Function	Arguments	Return Type	Description
CreateSkeletonRigC	<code>const char*</code> <code>rigName</code>	<code>UNIV::SkeletonRig*</code>	Returns a new heap allocated UNIV SkeletonRig
DeleteSkeletonRigC	<code>UNIV::SkeletonRig*</code> <code>ptr</code>	<code>bool</code>	Frees the memory reserved by the given heap allocated rig. Returns true if the operation was successful
AddBoneToSkeletonRigC	<code>UNIV::SkeletonRig*</code> <code>rig</code> , <code>float</code> <code>rotationX</code> , <code>float</code> <code>rotationY</code> , <code>float</code> <code>rotationZ</code> , <code>float</code> <code>rotationW</code> , <code>float</code> <code>positionX</code> , <code>float</code> <code>positionY</code> , <code>float</code> <code>positionZ</code> , <code>const char*</code> <code>boneName</code> , <code>int</code> <code>parentIndex</code> , <code>bool</code> <code>usingLocalValues</code> , <code>Utilities::StringContainer*</code> <code>utilities</code> , <code>errorMessage</code>	<code>bool</code>	Creates a bone and adds it to the given UNIV SkeletonRig. Returns true if the operation was successful

Function	Arguments	Return Type	Description
SetBoneTypeC	UNIV:: SkeletonBone * bone, uint32_t boneType	bool	Sets the bone type of the given UNIV SkeletonBone to the type provided in numeric form (see enums). Returns false if unsuccessful or if the bone already is of that type.
SetBoneTypeFromStringC	UNIV:: SkeletonBone * bone, const char* boneStr	bool	Same as the above function but uses the string form of the enum (ie "Default" and "Twist")
GetBoneTypeC	UNIV:: SkeletonBone * bone	uint32_t	Gets the bone type enum in numeric form
GetBoneTypeAsStringC	UNIV:: SkeletonBone * bone	const char*	Gets the bone type enum in string form

Function	Arguments	Return Type	Description
SetTwistBonePropertiesC	UNIV::SkeletonBone * bone, bool reassign, int32_t twistDriverIndex, float twistDriverWeight, Utilities::StringContainer* errorMessage	bool	Sets the twist bone properties of this bone, if reassign is set to true, the operation will redefine this bone as a twist bone if it isn't already. Returns if the operation was successful.
GetTwistBoneDriverIndexC	UNIV::SkeletonBone * bone, Utilities::StringContainer* errorMessage	int	Returns the index of twist bone driver
GetTwistBoneDriverWeightC	UNIV::SkeletonBone * bone, Utilities::StringContainer* errorMessage	float	Returns the amount of influence the drive has on this bone

Function	Arguments	Return Type	Description
SetMirrorIndexC	UNIV:: SkeletonBone * bone, int index	int	Calls the UNIV SkeletonBone's SetMirrorBoneIndex function. Returns the new index.
GetMirrorIndexC	UNIV:: SkeletonBone * bone	int	Returns the mirror bone index.
CreateBoneMirrorPairC	UNIV:: SkeletonRig * rig, int index1, int index2	bool	Pairs two bones to point to each other on their mirror index assignments.
ResetAllBoneMirrorsC	UNIV:: SkeletonRig * rig	bool	Resets all mirror indices on the UNIV SkeletonRig
VerifyExclusiveBoneMirrorsC	UNIV:: SkeletonRig * rig	bool	Returns true if any bone with a mirror index declared, has that mirror bone point back to it

Function	Arguments	Return Type	Description
GetSkeletonRigBoneCountC	UNIV:: SkeletonRig * source	size_t	Returns the total number of bone entries on this rig
GetSkeletonRigAnimatedBoneCountC	UNIV:: SkeletonRig * source	size_t	Returns the number of default bone entries on this rig (Twist bones, for example, are not typically animated and are not counted)

Function	Arguments	Return Type	Description
GetSkeletonRigNameC	UNIV:: SkeletonRig * source	<code>const char*</code>	Returns the name of the UNIV SkeletonRig
GetSkeletonBoneC	UNIV:: SkeletonRig * source, <code>int</code> index, Utilities:: StringContainer * errorMessage	UNIV:: SkeletonBone *	Returns a pointer to the UNIV SkeletonBone based on the given index
GetSkeletonBoneNameC	UNIV:: SkeletonBone * source	<code>const char*</code>	Returns the name of the given UNIV SkeletonBone
GetSkeletonBoneParentIndexC	UNIV:: SkeletonBone * source	<code>int</code>	Returns the parent index of the given UNIV SkeletonBone
GetSkeletonBoneRotationC	UNIV:: SkeletonBone * source, <code>bool</code> global	Math:: Quaternion *	Returns the rotation of the given UNIV SkeletonBone global = false will return relative to its parent
GetSkeletonBonePositionC	UNIV:: SkeletonBone * source, <code>bool</code> global	Math:: Vector3 *	Returns the position of the given UNIV SkeletonBone global = false will return relative to its parent
ValidateSkeletonRigNamesC	UNIV:: SkeletonRig * source, Utilities:: StringContainer * errorMessage	<code>bool</code>	Returns an expected type, with an errorMessage if has_value() is false. This function determines if all of the rig's bones have unique, case insensitive, names.

ValidateSkeletonRig ParentIndicesC	UNIV:: SkeletonRig * source, Utilities:: StringContainer * errorMessage	bool	Returns an expected type, with an errorMessage if has_value() is false. This function determines if all of the rig's bones have a valid parent index assigned.
---------------------------------------	--	----------------------	--

SkeletonBone

struct UNIV::SkeletonBone

Member Types

Member	Type	Description
localRotation	Math::Quaternion	Quaternion that represents the rotation relative to its parent bone
globalRotation	Math::Quaternion	Quaternion that represents the rotation relative to its rig origin
localPosition	Math::Vector3	Vector3 that represents the position of the bone relative to its parent bone
globalPosition	Math::Vector3	Vector3 that represents the position of the bone relative to its rig origin
name	Utilities::StringContainer	Name of this bone, should be unique

Member Functions

Function	Arguments	Return Type	Description
SetBoneTypeProperty	UNIV::BoneType boneType, bool resetExisting = false	bool	Returns true if the boneTypeProperty was set. If resetExisting is set to false, the bone will keep the property data if it is the same type as the desired type.
GetBoneTypeProperty	none	UNIV::BoneTypeProperties*	Gets this bone's type as a pointer to the base class. Casting will be needed to modify or access child class members
ResetBoneTypeProperty	UNIV::BoneType boneType = UNIV::BoneType ::Default	bool	A convenient way to set the Bone Type property with resetExisting set to true and default being automatically selected.
SetMirrorBoneIndex	int idx	void	Sets the mirrorBoneIndex for this bone. If no mirror is desired, -1 should be used.
ResetMirrorBoneIndex	none	void	Resets the mirrorBoneIndex to -1
GetMirrorBoneIndex	none	int	Returns the mirrorBoneIndex
SetParentBoneIndex	int idx	void	Sets the parentBoneIndex. Root is always 0 and points to a parent of -1

ResetParentBoneIndex	none	void	Sets the parentBoneIndex to Root (0)
SetAsRootBone	none	void	Sets the parentBoneIndex to -1, this does not affect the SkeletonRig itself as the index is determined by list order
GetParentBoneIndex	none	int	Returns the parentBoneIndex. If this bone has no parent, then either 0 or -1 will be returned.

BoneTypes

Bone Types is a system designed to store modular data about bones belonging to a rig. A bone may only have one type property at a time. Bone Types are meant to be universal and store game agnostic behavior. If a game specific property is required, see the RigPackages section, in which universal rigs can store multiple packages of game specific data for format conversion.

enum class UNIV::BoneType : uint32_t

Enum Values

Name	Value
Default	0
Twist	1
Undefined	0xFFFFFFFF (4294967295U)

struct UNIV::BoneTypeProperties (Abstract)

Base Class Members

Member	Type	Description
boneType	UNIV::BoneType	Default = UNIV::BoneType::Default

Base Class Functions

Function	Arguments	Return Type	Description
GetType			
GetTypeString			

struct UNIV::DefaultBoneProperties :
UNIV::BoneTypeProperties

Members

Member	Type	Description
boneType	UNIV::BoneType	Set to UNIV::BoneType::Default when constructed

struct UNIV::TwistBoneProperties :
UNIV::BoneTypeProperties

Members

Member	Type	Description
boneType	UNIV::BoneType	Set to UNIV::BoneType::Twist when constructed
twistDriverIndex	int32_t	-1 by default (no twist behavior is defined)
twistDriverWeight	float	Defines the amount of influence the bone at the given index has on this bone. Set to 0f by default

RigPackage

Overview

Rig Packages enable the user to add game or format specific data to a UNIV SkeletonRig without having to modify the universal structures. Each UNIV SkeletonRig has a package manager that tracks the packages via a vector of a base RigPackage type. Each package has a way to identify itself and as of now only one package of each type can be added to the manager's list. When future titles are supported, more packages may be introduced through their respective namespaces. Keeping the logic isolated and easier to expand upon.

struct UNIV::RigPackageManager

Member Types

Member	Type	Description
packages	Utilities::VectorContainer<UNIV::RigPackage*>	A collection or list of structs that inherit from the UNIV::RigPackage abstract struct

Member Functions

Function	Arguments	Return Type	Description
GetPackage	<code>const char*</code> packageName	Univ:: RigPackage *	Returns a pointer to the package requested. Nullptr is returned if none exists.
RemovePackage	<code>const char*</code> packageName	<code>bool</code>	Returns true if the package was successfully removed, however it is better to create a new package and overwrite the original if default data is desired. It is also advised to use the [GAME]::RigPackage specific instead, as memory is handled directly. A good example can be found here .
AddPackage	UNIV:: RigPackage * package, <code>bool</code> overwrite = <code>true</code>	<code>bool</code>	Returns true if a new package was successfully added. It is advised to use the functions provided by the specific [GAME]::RigPackage type instead, as memory is handled directly. A good example can be found here .

`struct` UNIV::[RigPackage](#) (abstract)

Member Functions

Function	Arguments	Return Type	Description
GetPackageType	none	<code>const char*</code>	Returns [GAME]_RIG_PACKA GE constant. Used when casting a RigPackage pointer or detecting if any of the UNIV rig's assigned packages are of this child type (there is only one allowed on each UNIV rig)

Utilities

Some utility classes are useful for compiler firewalling and are necessary to allow for certain STL functionality without breaking client applications.

WIP

To see information, take a look at the Utility header file. All of the currently implemented functionality is present. Due to the desire to have STL includes tucked away into the source files only, inline functions are to be avoided here. This means that template classes will need to be explicitly instantiated which can be done at their respective headers if desired. More functionality and instantiations will be added in the future.

External "C" Functions

Universal

- [Animation Scene Functions](#)
- [Animation Functions](#)
- [Animation Block Functions](#)
- [Animation Entry Functions](#)
- [Skeleton Rig Functions](#)

Starfield

- [Animation Scene Functions](#)
- [Rig Package Functions](#)